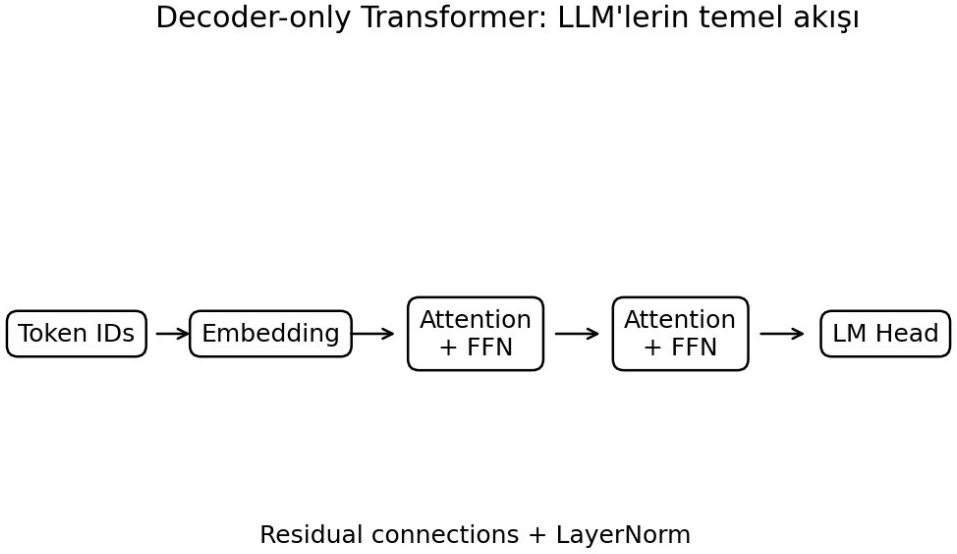
31. TRANSFORMERS & LLM

Attention • Tokenization • Hugging Face • Prompting • Fine-Tuning • RAG



Bu modülün amacı
Transformer ve büyük dil modeli (LLM) ekosistemini bir veri bilimci gözüyle anlamak: tokenlardan self-attention'a, pretrained model kullanımından prompting/fine-tuning'e ve RAG mimarisine kadar uçtan uca zihinsel model kurmak.

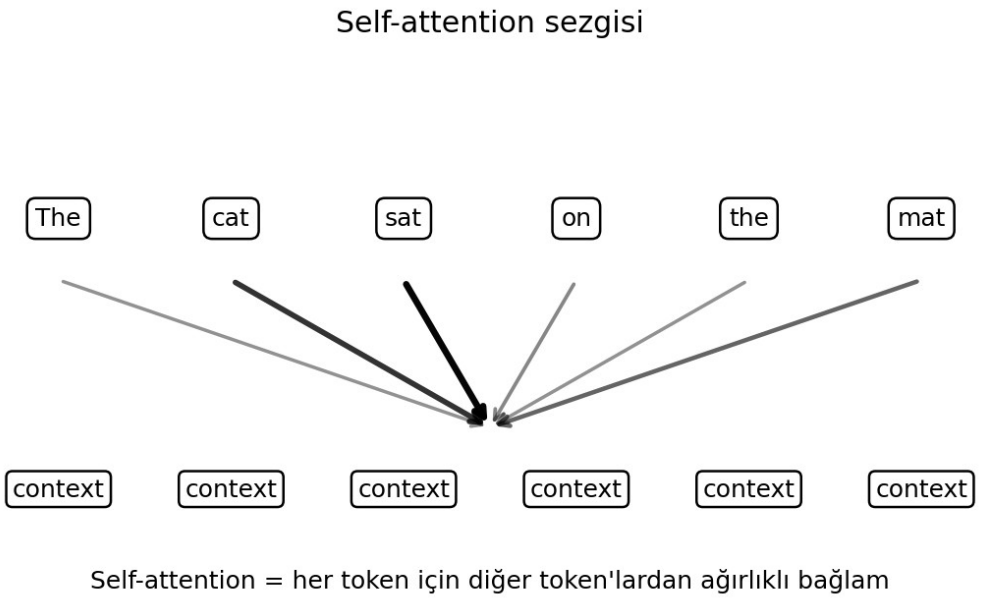
Kazanım	Modül sonunda
Transformer	Self-attention, multi-head attention ve encoder/decoder farkını açıklayabilirsin.
Tokenization	Token, vocabulary, context window ve embedding kavramlarını ayırabilirsin.
Hugging Face	Pretrained tokenizer/model ile inference ve fine-tuning akışını kurabilirsin.
Prompting	System/user prompt, few-shot ve structured output desenlerini tasarlayabilirsin.
Fine-tuning	SFT, LoRA/PEFT ve full fine-tune trade-off'larını bilirsin.
RAG	Retriever, vector search, context assembly ve generator ilişkisini kurabilirsin.

1. Transformer neden ortaya çıktı?

RNN/LSTM'ler sequence veriyi sırayla işler. Uzun bağımlılıklar, paralelleştirme ve büyük ölçekli training açısından sınırlamalar yaşanır. Transformer, sequence elemanları arasındaki ilişkileri attention üzerinden modelleyerek paralel training ve uzun bağlam temsilini güçlendirdi.

Yaklaşım	Ana fikir	Sınırlama / avantaj
RNN	Tokenları sırayla işler	Paralelleştirme zayıf
LSTM/GRU	Gated memory	Uzun context iyileşir ama sequential
Transformer	Attention ile tüm token ilişkileri	Paralel ve ölçeklenebilir

2. Attention sezgisi



Bir token'ın temsili oluşturulurken diğer token'ların ne kadar önemli olduğu öğrenilir. Bu önem ağırlıkları attention skorlarıdır.

Ana fikir
Attention 'hangi kelimeye bakmalıyım?' sorusunu differentiable bir mekanizma ile cevaplar.

3. Query, Key, Value

Parça	Sezgi
Query (Q)	Ben ne arıyorum?
Key (K)	Ben hangi bilgiyi temsil ediyorum?
Value (V)	Benim taşıdığım içerik ne?

Scaled dot-product attention özeti:

```
scores = (Q @ K.T) / sqrt(d_k)
weights = softmax(scores)
output = weights @ V
```

Neden sqrt(d_k)?
Dot-product büyüklüğünün dimension arttıkça aşırı büyümesini azaltarak softmax stabilitesine yardımcı olur.

4. Self-attention

Query, Key ve Value aynı input sequence'ten türetiliyorsa self-attention denir. Böylece her token diğer token'lardan bağlam toplar.

- Pronoun resolution
- Long-range dependencies
- Context-sensitive meaning
- Token-token interaction

Örnek
'bank' kelimesinin finans mı nehir kıyısı mı olduğunu çevresindeki tokenlar belirler.

5. Multi-head attention

Tek attention yerine paralel birden çok attention head farklı ilişki türlerini öğrenebilir.

Head olası rol	Örnek ilişki
Syntax	Subject-verb
Coreference	Pronoun -> entity
Position	Yakın/uzak token ilişkisi
Semantic	Konu / anlam ilişkisi

Not
Head'lerin insanlarca net bir role ayrıldığı garanti değildir; bu tablo sezgisel bir yorumdur.

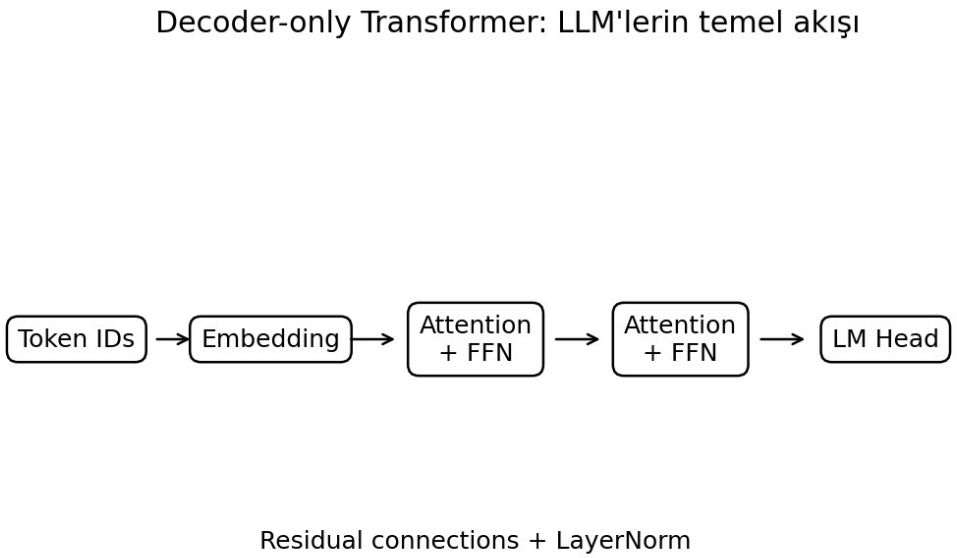
6. Positional information

Self-attention tek başına token sırasını bilmez. Positional encoding/embedding sıra bilgisini modele ekler.

- Absolute positional embedding
- Sinusoidal encoding
- Relative positions
- Rotary positional embeddings (RoPE)

LLM pratiği
Modern decoder-only LLM'lerde RoPE ve relative-position fikirleri çok yaygındır.

7. Transformer block



Bileşen	Görev
Attention	Tokenlar arası bilgi alışverişi
Feed-forward network	Token representation'ını nonlinear dönüştürür
Residual connection	Gradient akışını kolaylaştırır
LayerNorm	Training stabilitesi

8. Encoder vs Decoder

Mimari	Özellik	Tipik model
Encoder-only	Bidirectional context	BERT ailesi
Decoder-only	Causal next-token prediction	GPT benzeri LLM'ler
Encoder-decoder	Input -> output generation	T5/BART benzeri

Causal mask
Decoder-only model gelecekteki token'ı göremez; yalnız önceki tokenlara bakarak sıradaki token'ı tahmin eder.

9. Language modeling objective

Decoder-only LLM'lerin temel pretraining objective'i sıradaki token'ı tahmin etmektir.

```
P(token_t | token_1, ..., token_{t-1})
```

Basit objective, büyük sonuç
Çok büyük veri ve model ölçeğinde next-token prediction, dil bilgisi, dünya bilgisi ve görev çözme yeteneği gibi çok çeşitli davranışlar üretebilir.

10. Tokenization

LLM metni kelime olarak değil token id'leri olarak işler.

Kavram	Açıklama
Tokenizer	Metni tokenlara ve id'lere dönüştürür
Vocabulary	Modelin bildiği token seti
Subword	Kelime parçaları
Special tokens	BOS/EOS/PAD vb.
Token count	Maliyet/context hesabında kritik

Kelime != token
Türkçe gibi eklemeli dillerde tek kelime birden fazla subword token'a ayrılabilir.

11. BPE / WordPiece / SentencePiece

Yöntem	Kısa fikir
BPE	Sık karakter/subword çiftlerini birleştirir
WordPiece	Subword vocabulary üzerinden segmentasyon
SentencePiece	Whitespace'e bağımlı olmayan tokenization yaklaşımı

12. Embeddings

Token id'leri dense vector'lara dönüştürülür. Transformer blokları bu representation'ları bağlama göre sürekli günceller.

Embedding türü	Amaç
Token embedding	Token kimliği
Position embedding	Sıra bilgisi
Contextual hidden state	Bağlama göre güncellenmiş temsil
Sentence/document embedding	Semantic retrieval / clustering

13. Context window

Modelin tek inference sırasında işleyebildiği maksimum token sayısı context window'dur. Prompt, history, retrieved context ve output bu bütçeyi paylaşır.

Daha uzun her zaman daha iyi değil
Gereksiz context latency ve maliyeti artırabilir; önemli bilginin model tarafından etkili kullanılmasını da zorlaştırabilir.

14. Pretraining, instruction tuning, alignment

Aşama	Amaç
Pretraining	Genel dil/world representation
Instruction/SFT	Talimat takip etme
Preference/alignment	İstenen davranışa yönlendirme
Domain tuning	Belirli alan/görev performansı

15. Prompting temelleri

- Task açık yaz
- Girdi/çıktı formatını tanımla
- Gerekli context'i ver
- Örnek gerekiyorsa few-shot kullan
- Belirsiz kriterleri ölçülebilir hale getir
- Structured output gerekiyorsa schema belirt

Prompt engineering
Modelin parametrelerini değiştirmez; input aracılığıyla davranış yönlendirilir.

16. System, user, assistant rolleri

Rol	Amaç
System	Üst seviye davranış ve sınırlar
User	Görev ve kullanıcı girdisi
Assistant	Model cevabı / örnekler

17. Zero-shot, one-shot, few-shot

Tür	Tanım
Zero-shot	Örnek vermeden görev
One-shot	Bir örnek
Few-shot	Birkaç demonstrasyon

Few-shot ne sağlar?
Label formatını, karar mantığını ve output yapısını model için somutlaştırabilir.

18. Structured output

LLM cevabını JSON gibi makine-okunabilir bir formatta istemek production entegrasyonunu kolaylaştırır.

```
{
  "label": "refund_request",
  "confidence": 0.91,
  "reason": "Customer asks for refund"
}
```

Validation şart
LLM output'u external schema validator ile doğrulanmalı; parse edilemeyen output için retry/fallback tasarlanmalı.

19. Temperature, top-p ve decoding

Parametre	Etkisi
temperature	Randomness / dağılım keskinliği
top-p	Nucleus sampling
max tokens	Output sınırı
stop	Generation durdurma kuralları

Task bağımlı
Classification/extraction için daha deterministic; creative generation için daha açık sampling tercih edilebilir.

20. Hallucination nedir?

Modelin kulağa makul gelen fakat doğrulanmayan veya yanlış bilgi üretmesine hallucination denir.

- Bilgi model weights içinde eksik olabilir.
- Prompt belirsiz olabilir.
- Retrieved context yetersiz/yanlış olabilir.
- Model yanlış çıkarım yapabilir.

Çözüm tek teknik değil
RAG, citations, tool use, constrained output, verification ve human review birlikte kullanılabilir.

21. Hugging Face ekosistemi

Paket	Rol
transformers	Model/tokenizer API
datasets	Dataset load/map
tokenizers	Fast tokenization
accelerate	Device/distributed training
peft	LoRA/adapter fine-tuning
evaluate	Metric workflow

22. Pretrained model ile inference

```
from transformers import pipeline

clf = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

result = clf("The product is excellent.")
print(result)
```

Pipeline API
Hızlı prototip için güçlüdür; production/fine-tuning için tokenizer ve model sınıflarını ayrı yönetmek daha esnektir.

23. Tokenizer + model API

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(
    name,
    num_labels=2
)

batch = tokenizer(
    ["good product", "bad product"],
    padding=True,
    truncation=True,
    return_tensors="pt"
)

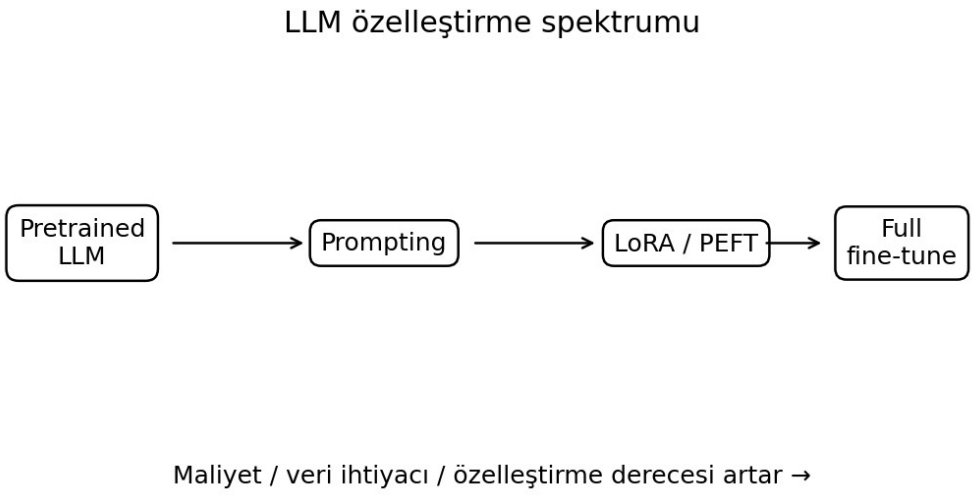
logits = model(**batch).logits
```

24. Attention mask

Padding tokenlarının gerçek input gibi attention'a katılmaması için attention mask kullanılır.

Tokenizer otomatik üretir
padding=True kullanılan çoğu Hugging Face tokenizer attention_mask döndürür.

25. Fine-tuning nedir?



Pretrained model ağırlıklarını belirli dataset/görev üzerinde güncelleyerek davranışı özelleştirmektir.

Yöntem	Avantaj	Maliyet
Prompting	En hızlı	Düşük
LoRA/PEFT	Az parametre train edilir	Orta
Full fine-tune	Maksimum kontrol	Yüksek

26. Supervised Fine-Tuning (SFT)

Instruction-input-output örnekleriyle modelin hedef davranışa adapte edilmesidir.

```
{
  "instruction": "Classify the ticket",
  "input": "I want my money back",
  "output": "refund_request"
}
```

Dataset kalitesi
Az ama temiz, tutarlı ve gerçek kullanım dağılımını temsil eden örnekler çoğu zaman çok değerlidir.

27. LoRA

LoRA, büyük weight matrix'leri doğrudan güncellemek yerine düşük-rank trainable adaptasyon matrisleri ekler.

Artı	Neden önemli?
Az trainable parameter	Daha düşük VRAM
Hızlı fine-tune	Experiment maliyeti düşer
Adapter saklama	Base model tekrar kullanılabilir
Multi-domain	Farklı adapter'lar yönetilebilir

28. PEFT

Parameter-Efficient Fine-Tuning, LoRA gibi yalnız küçük parameter alt kümesini train eden yöntem ailesidir.

Veri bilimci açısından
Büyük modeli sıfırdan eğitmek yerine pretrained model + PEFT çok daha gerçekçi bir iş akışıdır.

29. Quantization

Format	Kullanım
FP16/BF16	Training/inference memory düşürme
INT8	Inference memory/latency
4-bit	Büyük modeli düşük VRAM'de çalıştırma / QLoRA

Trade-off
Daha düşük precision memory ve hız avantajı sağlarken kalite kaybı veya kernel uyumluluğu riski yaratabilir.

30. QLoRA

Quantized base model üzerinde LoRA adapter'ları eğiterek büyük modelleri daha düşük VRAM ile fine-tune etmeyi amaçlar.

31. LLM evaluation

Task	Metric / yaklaşım
Classification	Accuracy/F1/PR-AUC
Extraction	Exact match / field F1
Generation	Human eval + task rubric
RAG QA	Answer correctness + retrieval relevance
Summarization	Human eval + factuality
Code	Unit tests / execution pass rate

LLM evaluation zordur
Tek otomatik metric çoğu generative görevde yeterli değildir. Rubric + human review + task-specific tests birlikte gerekir.

32. Golden dataset

Production LLM sistemi için gerçek kullanım örneklerinden oluşturulan küçük ama yüksek kaliteli evaluation set'i sürekli regression test için kullanılır.

- Edge cases
- Adversarial/ambiguous inputs
- Long-context examples
- Known failure modes

Prompt değişikliği de model değişikliği gibidir
Prompt sürümü değiştiğinde golden set üzerinde yeniden değerlendirme yap.

33. Latency ve cost

Faktör	Etkisi
Input tokens	Prompt/context maliyeti
Output tokens	Generation latency/maliyeti
Model size	Kalite vs latency
Batching	Throughput
Quantization	Memory ve hız
Caching	Tekrar sorgularda maliyet azaltır

34. KV cache

Autoregressive generation sırasında önceki tokenlar için key/value representation'larını tekrar hesaplamamak amacıyla cache tutulur.

Sonuç
Generation hızlanır ama context uzadıkça KV cache memory kullanımı artar.

35. Model serving

- Batching
- Streaming output
- Concurrency
- Timeout/retry
- Rate limits
- GPU memory
- Observability

Production KPI
Sadece model kalitesi değil p50/p95 latency, cost/request, error rate ve throughput da izlenir.

36. Prompt injection

Kullanıcı veya retrieved content modelin talimat hiyerarşisini manipüle etmeye çalışabilir.

- Untrusted content'i açıkça ayır
- Tool permissions minimum olsun
- Sensitive secrets prompt içine konmasın
- Output/action validation yap

RAG güvenliği
Retrieved doküman içeriği de güvenilmeyen input olarak ele alınmalıdır.

37. Tool use / function calling

LLM dış sistemlerde işlem yapmak için structured tool çağrısı üretebilir: database query, calculator, search, CRM lookup vb.

LLM karar verir, tool gerçeği getirir
Dinamik veya kesin bilgi gereken işlerde tool use, sadece model weights'e güvenmekten daha sağlamdır.

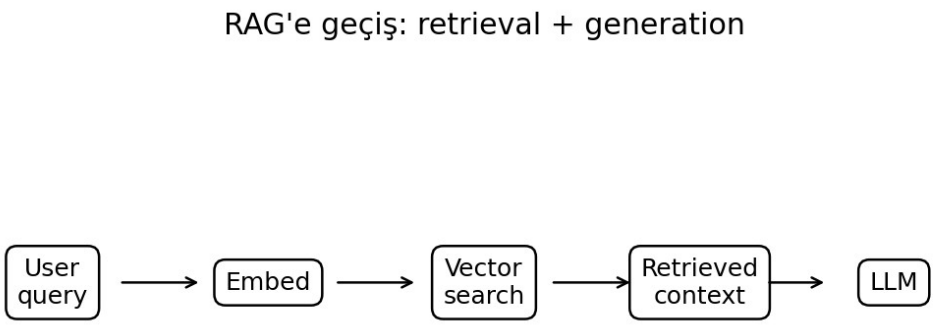
38. Agents

Agent yaklaşımı modelin bir hedef için birden fazla adım planlayıp tool çağrıları ve gözlemlerle ilerlemesidir.

Bileşen	Rol
LLM	Reasoning/decision
Tools	External actions/data
Memory/state	Adımlar arası bağlam
Controller	Loop, limits, stopping

Her probleme agent gerekmez
Basit deterministic pipeline daha ucuz, hızlı ve güvenilir olabilir.

39. RAG nedir?



Retriever external knowledge getirir; generator cevabı üretir.

Retrieval-Augmented Generation, kullanıcı sorusuna ilgili dış kaynak parçalarını getirip bunları LLM context'ine ekleyerek cevap üretir.

Neden?
Güncel/private bilgi, citation ihtiyacı ve hallucination riskini azaltmak için.

40. RAG bileşenleri

Bileşen	Görev
Chunking	Dokümanı parçalara ayır
Embedding	Semantic vector üret
Vector DB	Embedding sakla/search
Retriever	Relevant chunks getir
Reranker	Sonuçları yeniden sırala
Prompt builder	Query + context birleştir
LLM	Grounded answer üret

41. Chunking

Chunk çok büyükse retrieval noisy olabilir; çok küçükse anlam parçalanabilir. Chunk size/overlap doküman türüne göre validation ile seçilir.

Strateji	Örnek
Fixed tokens	500 token + 50 overlap
Paragraph	Paragraf bazlı
Heading-aware	Başlık/alt başlık yapısı
Semantic chunking	Anlam değişimine göre

42. Embedding ve semantic search

Query ve chunk'lar aynı embedding uzayına taşınır. Cosine similarity veya dot-product ile en yakın içerikler bulunur.

```
similarity = cosine(query_embedding, chunk_embedding)
```

32. modül
Vector database bölümünde FAISS/Chroma/Pinecone mantığı, indexing, metadata filtering ve retrieval sistemleri ayrıntılandırılacak.

43. Hybrid search

Dense semantic retrieval ile BM25/keyword retrieval birleştirilir.

Neden?
Exact product code, isim, hata kodu gibi lexical eşleşmeler dense embedding'de kaçabilir.

44. Reranking

İlk-stage retriever geniş candidate set getirir; cross-encoder/reranker daha pahalı ama daha hassas şekilde top sonuçları sıralar.

45. RAG evaluation

Katman	Soru
Retrieval	Doğru chunk geldi mi?
Context	Gereksiz/noisy context var mı?
Generation	Cevap doğru mu?
Grounding	Cevap context'e dayanıyor mu?
Citation	Kaynak iddiası doğru mu?

Sorunu katmanlara ayır
Cevap kötü ise hemen LLM'i suçlama; retrieval başarısızsa generator iyi context görmemiş olabilir.

46. Fine-tuning vs RAG

İhtiyaç	Tercih
Güncel/private facts	RAG
Belirli output format / style	Prompting veya SFT
Domain terminology	RAG + gerekirse fine-tune
Model davranışı	Fine-tuning
Kaynak gösterme	RAG
Knowledge sık değişiyor	RAG

Birbirinin alternatifi olmak zorunda değil
RAG + fine-tuned model birlikte kullanılabilir.

47. Gerçek vaka: support ticket routing

Katman	Çözüm
Input	Ticket text
Baseline	TF-IDF + Logistic Regression
Transformer	Encoder fine-tune
LLM	Few-shot structured classification
Metric	Macro-F1 + routing accuracy
Production	Confidence threshold + human fallback

48. Gerçek vaka: company knowledge assistant

Bileşen	Karar
Source	Policy/FAQ/internal docs
Index	Chunk + embedding + vector DB
Retrieval	Hybrid + metadata filter
Generation	Context-grounded prompt
Output	Answer + citations
Evaluation	Retrieval recall + grounded correctness

49. Gerçek vaka: structured extraction

Fatura/metin/email içinden belirli alanları JSON schema'ya çıkarmak.

LLM güçlü ama
Schema validation, confidence/fallback ve deterministic post-processing üretim sisteminde önemlidir.

50. Hangi yaklaşımı ne zaman?

Problem	Başlangıç yaklaşımı
Text classification	TF-IDF baseline -> encoder/LLM
Semantic search	Embedding + vector search
Knowledge QA	RAG
Style/format adaptation	Prompting -> SFT/LoRA
Dynamic actions	Tool use / workflow
Heavy reasoning workflow	Deterministic orchestration + gerekirse agent

51. Common anti-patterns

- Baseline kurmadan doğrudan en büyük LLM'e geçmek.
- Token budget ve latency ölçmemek.
- Prompt değişikliğini testsiz production'a vermek.
- RAG retrieval kalitesini ayrı ölçmemek.
- Fine-tuning ile sürekli değişen bilgiyi model weights'e gömmeye çalışmak.
- Structured output'u parse etmeyi modele bırakmak.
- Context'e gereksiz doküman doldurmak.
- Prompt injection riskini yok saymak.
- Evaluation set olmadan model kıyaslamak.
- Cost/request ve p95 latency izlememek.

52. Hugging Face fine-tuning iskeleti

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(name)

model = AutoModelForSequenceClassification.from_pretrained(
    name,
    num_labels=2
)

args = TrainingArguments(
    output_dir="runs/model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=3
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=valid_ds
)

trainer.train()
```

53. LLM project structure

```
llm_project/
  data/
    eval.jsonl
  src/
    prompts.py
    retrieval.py
    model.py
    evaluate.py
    guardrails.py
  configs/
    model.yaml
  tests/
    test_prompts.py
    test_schema.py
  notebooks/
  README.md
```

Prompt da source code'dur
Prompt versioning, tests ve evaluation artifact'ları repo içinde tutulmalı.

54. Mülakat soruları

Soru	Kısa cevap
Self-attention nedir?	Tokenların birbirinden ağırlıklı bağlam toplaması.
Q/K/V ne işe yarar?	Query arar, Key eşleşme temsil eder, Value içerik taşır.
Multi-head attention neden?	Farklı representation/ilişki altuzaylarını öğrenmek.
Positional encoding neden?	Attention sıra bilgisini kendi başına bilmez.
Encoder vs decoder?	Encoder bidirectional; decoder causal generation.
Tokenization neden subword?	Open vocabulary ve rare words yönetimi.
Context window nedir?	Tek inference'ta işlenen token bütçesi.
Temperature ne yapar?	Sampling randomness'ını etkiler.
LoRA nedir?	Low-rank trainable adapter fine-tuning.
RAG nedir?	Retrieval ile dış context getirip generation'a eklemek.
RAG vs fine-tuning?	Facts/güncellik için RAG; davranış için tuning.
Hybrid search?	Dense + lexical retrieval.
Reranker?	Candidate retrieval sonuçlarını daha hassas sıralar.
Hallucination?	Desteksiz/yanlış üretim.
Golden set?	Regression evaluation için sabit yüksek kaliteli örnek seti.

55. Uygulama alıştırmaları

1. Bir cümleyi tokenizer ile token id'lerine çevir ve token sayısını incele.
2. Hugging Face pipeline ile sentiment inference çalıştır.
3. Encoder-only ve decoder-only kullanım farkını açıklayan küçük deney yap.
4. Zero-shot ve few-shot classification promptlarını karşılaştır.
5. Aynı görevi farklı temperature değerleriyle test et.
6. Structured JSON output için schema validator ekle.
7. Küçük bir domain dataset üzerinde encoder fine-tune et.
8. LoRA/PEFT ile full fine-tune trade-off tablosu oluştur.
9. Basit embedding + cosine semantic search prototipi yap.
10. Mini RAG pipeline'ında retrieval failure ve generation failure örneklerini ayır.

56. Mini portföy projesi

Proje: Production-Oriented Internal Knowledge Assistant

Dosya	İçerik
src/chunking.py	Heading-aware chunking
src/embed.py	Document/query embeddings
src/retrieve.py	Vector + metadata retrieval
src/rerank.py	Optional reranking
src/prompt.py	Grounded answer prompt
src/evaluate.py	Retrieval + answer metrics
data/golden.jsonl	Evaluation queries
tests/	Schema/citation/edge-case tests
README.md	Architecture, cost, latency, limitations

CV cümlesi
Built a retrieval-augmented LLM assistant with embedding-based search, grounded prompting, citation-aware responses, golden-set evaluation and production-focused latency/cost monitoring.

57. Final cheat sheet

Kavram	Hatırlanacak fikir
Attention	Token ilişkilerini ağırlıklandır
Q/K/V	Match + content mechanism
Multi-head	Birden çok attention subspace
Encoder	Bidirectional understanding
Decoder	Causal generation
Tokenizer	Text -> token ids
Embedding	Dense representation
Context window	Token budget
Prompting	Weights değişmeden davranış yönlendirme
SFT	Supervised instruction tuning
LoRA/PEFT	Parameter-efficient tuning
Quantization	Memory/latency azaltma
RAG	External knowledge retrieval
Hybrid search	Dense + keyword
Reranking	Candidate quality artırma
Golden set	Regression evaluation
Tool use	External reliable actions/data

Bu modülden sonra
32. Vector Databases bölümünde embeddings, similarity search, FAISS/Chroma/Pinecone mantığı, indexing, metadata filtering ve RAG retrieval altyapısını ayrıntılı öğreneceksin.